

Executive Summary

The Redemption runs on Amazon EKS spread across three Availability Zones, sized for the steady baseline via an always-on managed node group and elastically expanded by Karpenter within seconds of a Flash Sale spike. Traffic reaches pods directly through an internet-facing ALB in IP-target mode, protected by AWS WAF. Every controller and the application itself authenticate to AWS purely through IRSA-scoped IAM roles — nodes themselves carry no broad permissions. A default-deny NetworkPolicy baseline, private subnets for all compute, and a private-first EKS control-plane posture implement defense in depth end to end. Reliability is enforced structurally: topology spread constraints and a PodDisruptionBudget guarantee the service survives an AZ outage or a node drain without dropping below safe capacity, while GitOps (ArgoCD) and progressive delivery remove the operational toil of manual rollouts.

A. Compute & Architecture

EKS 1.30 control plane with the API server privately reachable from the VPC and (for this assessment) publicly reachable for admin convenience — in production the public CIDR would be restricted to a corporate VPN range. Worker nodes run exclusively in private subnets across three AZs; only the ALB sits in public subnets. Data plane is split into two capacity pools:

- **Baseline managed node group** (min 3 / desired 6 / max 9, m6i.large/m6a.large) — always-on capacity sized for steady traffic, so the service never depends on cold-start provisioning for normal load.
- **Karpenter burst NodePool** — provisions on-demand nodes within seconds once pod scheduling pressure exceeds baseline capacity, and consolidates them back out 5 minutes after a spike passes.

The application Deployment uses **maxUnavailable: 0** on rolling updates, topology spread constraints across both AZ and node, and pod anti-affinity so a single AZ or node failure never removes more than a fraction of running capacity. The points ledger is stored in **DynamoDB** rather than a relational engine: it is a simple, high-throughput key-value/conditional-update workload, and DynamoDB's inherent multi-AZ durability and on-demand capacity mode absorb the 10x spike without any provisioning step.

B. Scalability Strategy

Scaling operates at two layers, both fully automatic:

- **Pod layer (HPA):** scales 6 → 90 replicas on CPU (60%), memory (70%), and a custom requests-per-second metric via the Prometheus Adapter — the RPS signal reacts to load the instant it arrives, rather than waiting for CPU to catch up. Scale-up has a 0s stabilization window and can double replica count every 30s; scale-down is deliberately conservative (5 min window, 20%/min) to avoid flapping once a Flash Sale ends.
- **Node layer (Karpenter):** watches for unschedulable pods and launches right-sized on-demand nodes (never spot, given the revenue-critical path) across all three AZs within seconds, capped at a hard 500 vCPU ceiling to bound worst-case spend.
- **Data layer:** DynamoDB PAY_PER_REQUEST mode scales throughput automatically with no capacity planning required for the 10x event.

C. Security & Networking

Least privilege: every controller (ALB controller, Karpenter, EBS CSI) and the application itself run under its own IRSA role scoped to only the AWS actions it needs — the app role, for example, is limited to read/write on its own DynamoDB table and GetSecretValue on its own secret path. Node IAM roles carry no application-level permissions.

Defense in depth: public subnets hold only the ALB and NAT gateways; nodes and pods live in private subnets with no direct route to the internet; the database tier sits in a third, isolated subnet group. A default-deny NetworkPolicy is applied cluster-wide in the redemption namespace, with explicit allow rules opening only ALB→pod (8080) and

Prometheus→pod (9090) ingress, and DNS/HTTPS egress. WAFv2 sits in front of the ALB for layer-7 protection, TLS 1.2+ is enforced at the listener, and all EC2/Karpenter nodes require IMDSv2.

D. Reliability & Observability

Health is measured through three separate probe types per pod — startup (covers slow init without false liveness failures), liveness (process-level health), and readiness (checks actual downstream DynamoDB connectivity, so a pod is pulled from rotation the moment it can't serve a deduction). kube-prometheus-stack provides cluster and application metrics with Grafana dashboards and Alertmanager routing; CloudWatch Container Insights serves as a secondary, AWS-native signal.

Failure-mode recovery is structural, not reactive:

- **AZ outage:** topology spread constraints and the 80% PodDisruptionBudget mean capacity is already distributed across the remaining AZs before the outage occurs — no manual failover step.
- **Bad deployment:** maxUnavailable: 0 rolling updates plus Argo Rollouts for canary analysis against error-rate/latency SLOs, with automatic rollback if the canary breaches thresholds.
- **Node loss:** the baseline managed node group and Karpenter both replace lost capacity automatically; the PDB prevents voluntary disruptions from compounding an involuntary one.

E. Operations

Day 2 Operations — minimizing toil

- **GitOps (ArgoCD)**: all cluster and application state is reconciled from Git — no manual kubectl apply against production.
- **Karpenter** removes manual node-group sizing decisions entirely for burst capacity.
- **EKS-managed add-ons** (VPC CNI, CoreDNS, kube-proxy, EBS CSI) offload patching of core cluster components to AWS.
- **Progressive delivery** (Argo Rollouts canaries) removes manual go/no-go judgment calls on every release.
- **Alerting tuned to SLOs** rather than raw thresholds, to reduce on-call noise.

Task Assignment — 3 engineers (1 Senior, 2 Juniors)

Engineer	Owns	Why
Senior	EKS cluster foundation (eks.tf, versions.tf), IRSA/security design (irsa.tf), Karpenter configuration, PR review gate on all IaC changes	Highest blast-radius decisions — cluster bootstrap and IAM boundary design set the security ceiling for everything built on top
Junior 1	Networking layer: VPC/subnets (vpc.tf), ALB Ingress + WAF wiring, NetworkPolicy manifests — under Senior review before merge	Well-bounded, testable in isolation (can be validated with connectivity tests) without needing full cluster context
Junior 2	Application layer: Deployment/Service/Ingress manifests, HPA/PDB tuning, kube-prometheus-stack + Grafana dashboard setup	Iterative and observable — mistakes surface quickly in dashboards rather than as silent security gaps

All three engineers work against the same Terraform repo via feature branches; the Senior is the required reviewer on eks.tf, irsa.tf, and karpenter.tf, while the networking and application manifests can be reviewed peer-to-peer once the Senior has approved the underlying cluster/IAM foundation.

Key Trade-offs

Decision	Trade-off accepted
DynamoDB over RDS for the points ledger	Gains hands-off scaling through the 10x spike and inherent multi-AZ durability; loses relational joins/ad-hoc querying — acceptable since the access pattern is a simple keyed ledger, not reporting.
On-demand-only Karpenter burst nodes (no Spot)	Costs more per burst node than Spot would; avoids the risk of Spot reclamation mid-Flash-Sale on a revenue-critical path.
Public EKS API endpoint left open in this assessment	Simplifies grading/demo access; production deployment should restrict cluster_endpoint_public_access_cidrs to a corporate VPN range.

Managed baseline node group + Karpenter burst, instead of Karpenter-only	Slightly more Terraform to maintain, but guarantees steady-state capacity is never dependent on Karpenter's provisioning latency.
--	---

Repository Contents

- **terraform/** — VPC, EKS cluster, IRSA roles, Karpenter, ALB Controller, DynamoDB, kube-prometheus-stack, ArgoCD
- **k8s/** — Namespace/ServiceAccount, Deployment, Service, Ingress, HPA, PodDisruptionBudget, NetworkPolicy, Karpenter NodePool
- **diagram/** — architecture diagram (data flow and AWS components)